[75.07 / 95.02 / TB025]

Algoritmos y programación III
Paradigmas y programación

Trabajo práctico 2: Into the Breach

Estudiantes:

Nombre	Padrón	Mail

Tutor:

Nota Final:

Índice

1. Objetivo 3
2. Consigna general 3
3. Especificación de la aplicación a desarrollar 4
3.1. Objetivo General del Juego 4
3.2. Componentes y Estructura del Tablero 4
3.3. Unidades 5
A. Mechs (controlados por el jugador) 5
B. Vek (controlados por el sistema) 5
3.4. Contaminación 5
3.5. Empuje y Colisiones 6
3.6. Estructura del Turno 6
3.7. Comportamiento de los Vek 6
3.8. Red Eléctrica 6
3.9. Condiciones de Victoria 7
4. Interfaz gráfica 8
5. Herramientas 8
6. Entregables 9
7. Formas de entrega 9
8. Evaluación 9
9. Entregables para cada fecha de entrega 10
Entrega 0 (Semana 13 del calendario) 10
Entrega 1 (Semana 14 del calendario) 10
Entrega 2 (Semana 15 del calendario) 11
Entrega 3 (Semana 16 del calendario) 11
Entrega 4 (A definir con el corrector) 11
10. Informe 12
Supuestos 12
Diagramas de clases 12
Diagramas de secuencia 12
Diagrama de paquetes 12
Detalles de implementación 12
Excepciones 12
Anexo 0: ¿Cómo empezar? 13
Requisitos, análisis 13
Anexo I: Buenas prácticas en la interfaz gráfica 13
Prototipo 13
JavaFX 13
Recomendaciones visuales 14
Tamaño de elementos 14
Contraste 14
Uso del color 14
Tipografía 14
Recomendaciones de interacción 15
Manejo de errores 15
Confirmaciones 15
Visibilidad del estado y otros 15

1. Objetivo

Desarrollar una aplicación de manera grupal aplicando todos los conceptos vistos en el curso, utilizando un lenguaje de tipado estático (Java) con un diseño del modelo orientado a objetos y trabajando con las técnicas de TDD e Integración Continua.

2. Consigna general

Desarrollar la aplicación completa, incluyendo el modelo de clases, sonidos e interfaz gráfica. La aplicación deberá ser acompañada por pruebas unitarias e integrales y documentación de diseño.

3. Especificación de la aplicación a desarrollar

3.1. Objetivo General del Juego

Into the Breach es un juego de táctica por turnos en el que el jugador controla un escuadrón de tres Mechs que defienden una ciudad del ataque de los Vek, criaturas gigantes que emergen del suelo. Su característica central es que todos los ataques enemigos se anuncian antes de ejecutarse: el jugador ve qué casilla atacará cada Vek y debe reaccionar moviendo, atacando o empujando unidades para evitar el daño. La partida consiste en una única misión. El jugador gana si sobrevive todos los turnos de la misión sin que la Red Eléctrica llegue a 0.

3.2. Componentes y Estructura del Tablero

La misión se juega en un tablero de 8x8 casillas. Cada casilla tiene un tipo y puede estar ocupada por, a lo sumo, una unidad.

Tipo de casilla	Descripción
Suelo	Casilla normal. Se puede transitar y ocupar.
Edificio	Obstáculo con 1 punto de vida. Al recibir daño resta 1 punto a la Red Eléctrica y queda destruido (pasa a ser Suelo).
Montaña	Obstáculo indestructible. Bloquea el movimiento y los disparos.
Agua	Se puede transitar. Un Vek terrestre que termina en el Agua (por movimiento o empuje) muere. Un Mech en el Agua no puede atacar. Las unidades voladoras no se ven afectadas.
Ácido	Se puede transitar. La unidad terrestre que termina en esta casilla (por movimiento o empuje) queda Contaminada (ver 3.4). Las unidades voladoras no se ven afectadas.

La disposición del tablero (tipos de casilla, posición inicial de las unidades y el orden en que emergerán los nuevos Vek) se carga a partir de un mapa predefinido.

3.3. Unidades

A. Mechs (controlados por el jugador)

El escuadrón tiene tres Mechs, todos terrestres. En cada turno, cada Mech puede moverse y luego atacar, o realizar solo una de las dos acciones.

Mech	Vida	Movimiento	Arma	Efecto del arma
Mech de Combate	3	3	Puño	Golpea una casilla adyacente: 2 de daño y empuja al objetivo 1 casilla en la dirección del golpe.
Mech Cañón	3	3	Cañón	Dispara en línea recta: impacta a la primera unidad u obstáculo en esa dirección. 1 de daño y empuje de 1 casilla.
Mech de Artillería	2	3	Artillería	Dispara en arco (ignora obstáculos intermedios) a una casilla en línea recta a 2 o más casillas: 1 de daño en esa casilla y empuja hacia afuera a las unidades de las 4 casillas adyacentes.

El movimiento se mide en casillas ortogonales y no puede atravesar obstáculos ni otras unidades. Un Mech con vida 0 queda destruido y se retira del tablero.

B. Vek (controlados por el sistema)

Vek	Vida	Movimiento	Ataque
Saltador	2	3	Muerde una casilla adyacente: 2 de daño.
Luciérnaga	3	2	Escupe ácido en línea recta: 1 de daño a la primera unidad u obstáculo. Si impacta a una unidad, la deja Contaminada.
Avispón (volador)	2	4	Pica una casilla adyacente: 1 de daño. Puede volar por encima de obstáculos y otras unidades, pero debe terminar su movimiento en una casilla libre.

El Saltador y la Luciérnaga son terrestres. El Avispón es volador: el Agua y el Ácido no lo afectan.

3.4. Contaminación

Una unidad Contaminada recibe el doble de todo el daño (de ataques y de choques) hasta el final de la misión. La contaminación no se acumula: una unidad ya Contaminada que vuelve a contaminarse sigue recibiendo el doble de daño. Los edificios y las montañas no pueden contaminarse.

3.5. Empuje y Colisiones

El empuje es la mecánica principal del juego. Al empujar una unidad 1 casilla:

1. Si la casilla destino está libre (Suelo, Agua o Ácido), la unidad se desplaza y se aplica el efecto de la casilla.
2. Si la casilla destino tiene otra unidad u obstáculo, la unidad empujada no se mueve y recibe 1 de daño. Si el obstáculo es otra unidad o un edificio, también recibe 1 de daño (el edificio resta Red Eléctrica). La montaña no recibe daño.
3. Si la casilla destino está fuera del tablero, la unidad no se mueve.

Importante: el ataque anunciado de un Vek queda fijado en dirección relativa a su posición. Si el Vek es empujado, su ataque se desplaza con él. Si queda apuntando a otro Vek, lo daña.

3.6. Estructura del Turno

La misión dura 5 turnos. Cada turno sigue la siguiente secuencia:

1. **Fase Vek:** cada Vek vivo se mueve y anuncia la casilla que atacará. El sistema marca también la casilla donde emergerá el próximo Vek.
2. **Fase del Jugador:** el jugador activa sus Mechs en el orden que elija. Cada Mech puede moverse y atacar una vez.
3. **Fase de Resolución:** los ataques anunciados se ejecutan en el orden en que fueron anunciados. Un Vek muerto antes de su turno no ataca.
4. **Emergencia:** aparece el nuevo Vek en la casilla marcada. Si una unidad ocupa esa casilla, bloquea la emergencia y recibe 1 de daño.

Deshacer movimiento: durante la Fase del Jugador, una vez por turno, el jugador puede deshacer el movimiento de un Mech que todavía no atacó. El Mech vuelve a su posición anterior, se revierten todos los efectos de ese movimiento (por ejemplo, la contaminación por haber terminado sobre Ácido) y puede volver a moverse.

3.7. Comportamiento de los Vek

La IA de los Vek debe ser determinista, para que sus decisiones puedan probarse. Cada Vek evalúa las siguientes reglas en orden y aplica la primera que pueda cumplir:

Prioridad	Regla
1	Moverse a una posición desde la que pueda atacar un Edificio.
2	Moverse a una posición desde la que pueda atacar un Mech.
3	Acercarse lo más posible al edificio más cercano, sin anunciar ataque.

Los Vek terrestres nunca terminan su movimiento en el Agua ni en el Ácido. Los empates se resuelven con un criterio fijo documentado en los Supuestos (por ejemplo, la casilla con menor fila y luego menor columna).

3.8. Red Eléctrica

La Red Eléctrica representa la energía de la ciudad. Empieza en 5 puntos y cada punto de daño a un edificio resta 1 punto. Si llega a 0, la partida se pierde.

3.9. Condiciones de Victoria

El sistema verifica las condiciones de fin de partida al final de cada Fase de Resolución:

Condición	Resultado
Se completan los 5 turnos con la Red Eléctrica mayor a 0.	Victoria
La Red Eléctrica llega a 0.	Derrota
Los tres Mechs son destruidos.	Derrota

Al terminar la partida, el sistema muestra el resultado, la cantidad de Vek eliminados y la Red Eléctrica final.

4. Interfaz gráfica

La interacción entre el usuario y la aplicación deberá ser mediante una interfaz gráfica intuitiva. Consistirá en una aplicación de escritorio utilizando JavaFX y se pondrá mucho énfasis y se evaluará como parte de la consigna su usabilidad. (*en el anexo I se explicarán algunas buenas prácticas para armar la interfaz gráfica de usuario o GU*I)

Como este juego se basa en anticipar las acciones del enemigo, la interfaz deberá mostrar en todo momento:

- Las casillas que atacará cada Vek y el orden en que se resolverán los ataques.
- La casilla donde emergerá el próximo Vek.
- Las casillas alcanzables al seleccionar un Mech y los objetivos válidos al apuntar su arma.
- La vida de cada unidad, qué unidades están Contaminadas, la Red Eléctrica y el turno actual sobre el total.
- Si la opción de deshacer movimiento está disponible en el turno actual.

5. Herramientas

1. JDK (Java Development Kit): Versión 1.8 o superior.
2. JavaFX
3. JUnit 5 y Mockito: Frameworks de pruebas unitarias para Java.
4. IDE (Entorno de desarrollo integrado): Su uso es opcional y cada integrante del grupo puede utilizar uno distinto o incluso el editor de texto que más le guste. Lo importante es que el repositorio de las entregas no contenga ningún archivo de ningún IDE y que la construcción y ejecución de la aplicación sea totalmente independiente del entorno de desarrollo.
    a. Eclipse
    b. IntelliJ
    c. Netbeans
5. Herramienta de construcción: Se deberán incluir todos los archivos XML necesarios para la compilación y construcción automatizada de la aplicación. El informe deberá contener instrucciones acerca de los comandos necesarios (preferentemente también en el archivo README.md del repositorio). Puede usarse Maven o Apache Ant con Ivy.
6. Repositorio remoto: Todas las entregas deberán ser subidas a un repositorio único en GitHub para todo el grupo en donde quedarán registrados los aportes de cada miembro. El repositorio puede ser público o privado. En caso de ser privado debe agregarse al docente corrector como colaborador del repositorio.
7. Git: Herramienta de control de versiones
8. Herramienta de integración continua: Deberá estar configurada de manera tal que cada *commit* dispare la compilación, construcción y ejecución de las pruebas unitarias automáticamente. Algunas de las más populares son:
    a. Travis-CI
    b. Jenkins
    c. Circle-CI
    d. GitHub Actions (recomendado)

Se recomienda basarse en la estructura del proyecto base armado por la cátedra.

6. Entregables

Para cada entrega se deberá subir lo siguiente al repositorio:

1. Código fuente de la aplicación completa, incluyendo también: código de la prueba, archivos de recursos.
2. Script para compilación y ejecución (Ant o Maven).
3. Informe, acorde a lo especificado en este documento (en las primeras entregas se podrá incluir solamente un enlace a Overleaf o a Google Docs en donde confeccionen el informe e incluir el archivo PDF solamente en la entrega final).

No se deberá incluir ningún archivo compilado (formato .class) ni tampoco aquellos propios de algún IDE (por ejemplo .idea). Tampoco se deberá incluir archivos de diagramas UML propios de alguna herramienta. Todos los diagramas deben ser exportados como imágenes de manera tal que sea transparente la herramienta que hayan utilizado para crearlos.

7. Formas de entrega

Habrá 4 entregas formales que tendrán una calificación de APROBADO o NO APROBADO en el momento de la entrega. Además, se contará con una entrega 0 preliminar.

Aquel grupo que acumule 2 no aprobados, quedará automáticamente desaprobado con la consiguiente pérdida de regularidad en la materia de todos los integrantes del grupo. *En cada entrega se deberá incluir el informe actualizado*.

8. Evaluación

El día de cada entrega, cada ayudante convocará a los integrantes de su grupo, solicitará el informe correspondiente e iniciará la corrección mediante una entrevista grupal. Es imprescindible la presencia de todos los integrantes del grupo el día de cada corrección.

Se evaluará el trabajo grupal y a cada integrante en forma individual. El objetivo de esto es comprender la dinámica de trabajo del equipo y los roles que ha desempeñado cada integrante del grupo. Para que el alumno apruebe el trabajo práctico debe estar aprobado en los dos aspectos: grupal e individual (se revisarán los commits de cada integrante en el repositorio).

Dentro de los ítems a chequear el ayudante evaluará aspectos formales (como ser la forma de presentación del informe), aspectos funcionales: que se resuelva el problema planteado y aspectos operativos: que el TP funcione integrado.

9. Entregables para cada fecha de entrega

Cada entrega consta de las pruebas + el código que hace pasar dichas pruebas.

Cada entrega presupone que los equipos realizarán refactor de su código según crean conveniente.

Entrega 0 (Semana 12 del calendario)

- Planteo de modelo tentativo.
- Diagrama de clases general.
- Diagrama de secuencia para el caso donde el Mech de Artillería dispara a un Vek que está al lado de un edificio: daño en la casilla objetivo, empuje de las unidades adyacentes, choque contra el edificio y descuento de la Red Eléctrica.

Entrega 1 (Semana 13 del calendario)

Pruebas (sin interfaz gráfica):

- Verificar que el tablero se cargue con 8x8 casillas a partir de un mapa predefinido, con los tipos de casilla en las posiciones correctas.
- Verificar que un Mech solo pueda moverse a casillas dentro de su rango de movimiento, sin atravesar obstáculos ni otras unidades.
- Verificar que el Puño cause 2 de daño a una unidad adyacente y la empuje 1 casilla en la dirección del golpe.
- Verificar que el Cañón impacte a la primera unidad u obstáculo en línea recta y no a las que están detrás.
- Verificar que la Artillería pueda atacar por encima de obstáculos, que no pueda atacar casillas adyacentes y que dañe la casilla objetivo y empuje hacia afuera a las unidades de las 4 casillas adyacentes.
- Verificar que un Mech no pueda moverse ni atacar dos veces en el mismo turno.
- Verificar que se pueda deshacer el movimiento de un Mech que todavía no atacó y que vuelva a su posición anterior.
- Verificar que no se pueda deshacer un movimiento después de que el Mech atacó, ni más de una vez por turno.

Entrega 2 (Semana 14 del calendario)

Pruebas (sin interfaz grafica):

- Verificar que al empujar una unidad contra otra unidad u obstáculo, la unidad no se mueva y se aplique el daño de colisión.
- Verificar que el daño a un edificio reduzca la Red Eléctrica en 1 punto.
- Verificar que un Vek terrestre empujado al Agua muera, que el Avispón no, y que un Mech en el Agua no pueda atacar.
- Verificar que una unidad terrestre que termina sobre Ácido quede Contaminada y reciba el doble de daño, y que el Avispón no se contamine.
- Verificar que la Luciérnaga contamine a la unidad que impacta y que la contaminación no se acumule.
- Verificar que al deshacer el movimiento de un Mech que terminó sobre Ácido, el Mech deje de estar Contaminado.
- Verificar que cada Vek anuncie su ataque en la Fase Vek y que se ejecute en la Fase de Resolución, en el orden anunciado.
- Verificar que al empujar a un Vek su ataque anunciado se desplace con él, y que si queda apuntando a otro Vek, lo dañe.
- Verificar que la IA de los Vek aplique las reglas de prioridad en orden.
- Verificar que una unidad sobre la casilla de emergencia bloquee la aparición del nuevo Vek y reciba 1 de daño.
- Planteo inicial de interfaz gráfica (mockups/dibujos) mostrando: el tablero con los ataques anunciados, la información de cada unidad, la Red Eléctrica y el turno actual.

Entrega 3 (Semana 15 del calendario)

- Modelo del juego terminado.
- Verificar las condiciones de victoria y derrota: completar los 5 turnos, Red Eléctrica en 0 y los tres Mechs destruidos.
- Interfaz gráfica inicial básica: comienzo del juego, visualización del tablero con los ataques anunciados e interfaz de usuario básica para mover, atacar y deshacer movimiento.
- Modelo del manejo de turnos: secuencia correcta de Fase Vek, Fase del Jugador, Fase de Resolución y Emergencia.

Entrega 4 (Semana 16 del calendario)

Trabajo Práctico completo funcionando, con interfaz gráfica final, **sonidos** e informe completo.

Tiempo total de desarrollo del trabajo práctico:

5 semanas

10. Informe

El informe deberá estar subdividido en las siguientes secciones:

Supuestos

Documentar todos los supuestos hechos sobre el enunciado. Asegurarse de validar con los docentes.

Diagramas de clases

Varios diagramas de clases, mostrando la relación estática entre las clases. Pueden agregar todo el texto necesario para aclarar y explicar su diseño de manera tal que el modelo logre comunicarse de manera efectiva.

Diagramas de secuencia

Varios diagramas de secuencia, mostrando la relación dinámica entre distintos objetos planteando una gran cantidad de escenarios que contemplen las secuencias más interesantes del modelo.

Diagrama de paquetes

Incluir un diagrama de paquetes UML para mostrar el acoplamiento de su trabajo.

Detalles de implementación

Deben detallar/explicar qué estrategias utilizaron para resolver todos los puntos más conflictivos del trabajo práctico. Justificar el uso de herencia vs. delegación, mencionar que principio de diseño aplicaron en qué caso y mencionar qué patrones de diseño fueron utilizados y por qué motivos.

> IMPORTANTE
>
> No describir el concepto de herencia, delegación, principio de diseño o patrón de diseño. Solo justificar su utilización.

Excepciones

Explicar las excepciones creadas, con qué fin fueron creadas y cómo y dónde se las atrapa explicando qué acciones se toman al respecto una vez capturadas.

Anexo 0: ¿Cómo empezar?

Requisitos, análisis

¿Cómo se empieza? La respuesta es lápiz y papel. No código!.

1. Entiendan el dominio del problema. Definir y utilizar un lenguaje común que todo el equipo entiende y comparte. Ej.: Si hablamos de “X entidad”, todos entienden que es algo ... Si los conceptos son ambiguos nunca podrán crear un modelo congruente.
2. Compartan entre el grupo, pidan opiniones y refinen la idea en papel antes de sentarse a programar.

Anexo I: Buenas prácticas en la interfaz gráfica

El objetivo de esta sección es recopilar algunas recomendaciones en la creación de interfaces gráficas de usuario o GUI. La idea no es exigir un diseño refinado y prolijo, sino que pueda ser usado por el grupo de docentes de Algo3 sin impedimentos “lo mejor posible”.

Prototipo

Venimos escribiendo código, integrales y UML todo el cuatrimestre. Me están pidiendo una interfaz gráfica. ¿Cómo se empieza? La respuesta es lápiz y papel. No código!.

1. Armen un dibujo o prototipo en papel (pueden usar google docs o cualquier herramienta también) de todas las “pantallas”. No tiene que ser perfecto.
2. Compartan entre el grupo, pidan opiniones y refinen la idea en papel antes de sentarse a programar.

JavaFX

Entender cómo funciona JavaFX es clave para implementar la GUI correctamente. No subestimen el tiempo que lleva implementar y modificar la UI o interfaz de usuario. Lean todo lo que ofrece y las buenas prácticas de la tecnología. Hay plugins específicos para el IDE, pero por más que usen herramientas WYSIWYG, siempre conviene entender la API para mejorar el código autogenerado que casi nunca es óptimo.

Recomendaciones visuales

Tamaño de elementos

- Se recomienda un tamaño de tipografía de al menos a 10 puntos al mayor contraste negro contra blanco para asegurar la legibilidad, o bien 12 puntos.
- Para los botones o elementos interactivos, el tamaño mínimo del área debería ser de 18x18.
- Extra: Los elementos más importantes deberían ser más grandes y estar en posiciones más accesibles (poner ejemplos)

Contraste

- Aseguren que el texto y los elementos tengan buen contraste y se puedan leer bien
- Se sugiere un contraste cercano a 3 entre textos y fondos para texto grande e imágenes.
- Herramientas para verificar contraste: https://colourcontrast.cc/ https://contrast-grid.eightshapes.com

Uso del color

- La recomendación es no utilizar más de 3 colores para la UI y los elementos
- En el enunciado se ejemplifica con una paleta accesible, pero pueden usar cualquiera para las fichas
- Accesibilidad: Si usan para las fichas rojo y verde, o verde y azul, aseguren que las personas con daltonismo puedan distinguirlas usando letras o símbolos sobre las mismas.
- Dudas eligiendo paletas? https://color.adobe.com

Tipografía

- No se recomienda usar más de 2 tipografías para toda la aplicación
- Asegurarse de que esas tipografías se exporten correctamente en en TP
- Evitar tipografías “artísticas” para texto, menú y botones, ya que dificultan la lectura

Recomendaciones de interacción

Manejo de errores

- No escalar excepciones a la GUI. Es un No absoluto. Enviar a la consola.
- Si muestran errores al usuario, el mensaje de error debe estar escrito sin jerga técnica y permitir al usuario continuar y entender lo que está pasando
- Siempre optar por validar y prevenir errores, a dejar que el usuario ejecute la acción y falle.

Confirmaciones

- Antes de cerrar o ejecutar cualquier operación terminal, una buena práctica es pedir confirmación al usuario (para el alcance del tp no sería necesario)

Visibilidad del estado y otros

- Mostrar el estado en el cual está el juego. Siempre debería estar accesible
- Permitir al usuario terminar o cerrar en cualquier momento